```python
import numpy as np

class Module:
    def update(self,lr): pass
    def average(self,nn,a): pass
    def backward(self,DY): pass
    def forward(self,X): pass

class Sequential(Module):

    def __init__(self,modules):
        self.modules = modules

    def forward(self,X):
        '''Given input X, perform a full forward pass through the MLP'''
        #TODO: fill in this function
        for m in self.modules:
            X = m.forward(X)
        return X

    def backward(self,DY):
        '''Perform a full backward pass through the MLP.
            DY is gradient of the loss w.r.t the final output'''
        #TODO: fill in this function
        for m in reversed(self.modules):
            DY = m.backward(DY)

        return DY

    def update(self,lr):
        for m in self.modules:
            X = m.update(lr)

    def average(self,nn,a):
        for m,n in zip(self.modules,nn.modules):
```

```
36              m.average(n,a)
37
38 class Input(Module):
39     def __init__(self, inp):
40         R, Z = inp
41         sample_in = np.concatenate([R, np.expand_dims
   (Z, -1)], axis = -1)
42         self.nbout = sample_in.shape[-2] * sample_in.
   shape[-1]
43
44     def forward(self,inp):
45         R, Z = inp
46         rz = np.concatenate([R, np.expand_dims(Z, -1
   )], axis = -1)
47         return rz.reshape(rz.shape[0], -1)
48
49 class Output(Module):
50
51     def __init__(self,T):
52         self.tmean = T.mean()
53         self.tstd  = T.std()
54         self.nbinp = 1
55
56     def forward(self,X):
57         # un-normalize the final prediction
58         self.X = X.flatten()
59         return self.X*self.tstd+self.tmean
60
61     def backward(self,DY):
62         #TODO: fill in this function
63         return DY * self.tstd
64
65 class Linear(Module):
66
67     def __init__(self,m,n):
68         self.lr = 1 / np.sqrt(m)
69         self.W = np.random.normal(0,1 / m**.5,[m,n]).
   astype('float32')
```

```
 70         self.B = np.zeros([n]).astype('float32')
 71
 72     def forward(self,X):
 73         #TODO: fill in this function
 74         self.input = X
 75         out = np.dot(X, self.W) + self.B
 76         if np.isnan(out).any():
 77             print("NaN detected in Linear.forward")
 78         return np.dot(X, self.W) + self.B
 79
 80     def backward(self,DY):
 81         #TODO: fill in this function
 82         DY = DY.reshape(-1, self.W.shape[1])
 83         self.DW = np.dot(self.input.T, DY)
 84         self.DB = np.sum(DY, axis=0)
 85         return np.dot(DY, self.W.T)
 86
 87     def update(self,lr):
 88         self.W -= lr*self.lr*self.DW
 89         self.B -= lr*self.lr*self.DB
 90
 91     def average(self,nn,a):
 92         self.W = a*nn.W + (1-a)*self.W
 93         self.B = a*nn.B + (1-a)*self.B
 94
 95 class Tanh(Module):
 96
 97     def forward(self,X):
 98         #TODO: fill in this function
 99         self.input = X
100         return np.tanh(X)
101     def backward(self,DY):
102         #TODO: fill in this function
103         return DY * (1-np.tanh(self.input)**2)
104
105
```